Programming Assignment 2

Sanoj S Vijendra
22b0916

Table of Content

Task 1 (MDP Planning):

1. **Algorithms:**
    a. **Howard's Policy Iteration:**
    Core Idea
    π ← Arbitrary policy.
    While π has improvable states:
      π' ← PolicyImprovement(π)
      π ← π'
    Return π
    b. **Linear Programming:**
    Core Idea:
    Maximise (- $\Sigma_{s \in S} V(s)$)
    subject to:
    $V(s) \geq \Sigma_{s' \in S} T(s, a, s')\{R(s, a, s') + \gamma V(s')\}, \forall s \in S, a \in A.$

2. Core Design and Assumptions:

a. **Data Structures**: The MDP's transition function, which can be sparse, was stored as a list of dictionaries ([state][action] -> {next_state: (reward, prob)}). This is memory efficient and fast for iterating over the possible outcomes of a specific state action pair.
b. **Assumptions**:
    i. Input Validity: The input MDP file is assumed to be correctly formatted as per the specification.
    ii. Terminal States: The value of any terminal (end) state is defined to be zero. No further rewards can be accumulated from these states.
    iii. Tie Breaking: During the policy improvement step, if multiple actions yield the same maximum Q-value, the action with the lowest numerical index is selected, consistent with the default behavior of *numpy.argmax*.

3. Algorithm Implementation and Optimization:

a. Howard's Policy Iteration

The implementation of HPI involved iterating between two steps: Policy Evaluation and Policy Improvement. The performance of this algorithm was critically dependent on the efficiency of the Policy Evaluation step.

1. Initial Approach (Exact Solver): The first implementation of Policy Evaluation formulated the Bellman expectation equations as a system of $|S|$ linear equations and solved it using *numpy.linalg.solve()*. This method was found to be exceptionally slow, with a time complexity of roughly

$O(S^3)$. It does not exploit the natural sparsity of MDP transition matrices, making it impractical for state spaces larger than a few hundred states.

2. Second Approach (Iterative Evaluation): The next iteration replaced the exact solver with an iterative approach that repeatedly swept through all states, updating their values until they converged (till tolerance 10e-8). While algorithmically superior for sparse problems, its implementation in pure Python loops was still slower than expected due to Python's inherent overhead compared to optimized, low level code.
3. Final Approach (Modified Policy Iteration): The key optimization was to implement Modified Policy Iteration. Instead of waiting for the value function to fully converge in the evaluation step, we limited it to a small, fixed number of iterations (e.g., 30). This provides a "good enough" value estimate to find an improving policy.

b. Linear Programming

The LP formulation was implemented using the PuLP library. LP was chosen as the default algorithm.

4. Performance Observations: HPI vs LP

A significant observation was practical performance of Howard's Policy Iteration (HPI) against a Linear Programming (LP) formulation. The initial observation was that the LP solver was significantly faster than the naive HPI implementation that used numpy.linalg.solve(). This was expected, as the LP solver (like the default CBC solver used by PuLP) is a highly optimized C++ program designed for large, sparse systems, whereas the np.linalg.solve routine has a debilitating $O(S^3)$ complexity for dense matrices.

Following this, the HPI algorithm was substantially improved by replacing the exact solver with Modified Policy Iteration. As shown in the final code, this involves running the policy evaluation step for a fixed number of iterations (30 in this case). This correctly replaces the $O(S^3)$ bottleneck with a much more efficient algorithm that scales with the number of transitions, not the cube of the number of states.

However, the final and most critical observation was that even this algorithmically superior HPI implementation remained consistently slower than the LP solver. The reason for this outcome lies not in the algorithm's theory but in its execution environment. The HPI implementation relies on standard Python for loops to iterate through states and transitions. As an interpreted language, Python carries significant overhead for such loops, especially when the state space is large. In contrast, the LP solver is a pre-compiled, low-level program that executes these types of sparse calculations orders of magnitude faster than the Python interpreter.

This leads to a crucial conclusion the raw computational speed of the compiled C++ backend of the LP solver outweighed the algorithmic efficiency of the Python-based HPI. While HPI is an excellent algorithm for MDPs, its performance in this case was

fundamentally limited by the implementation language. The LP approach, despite being a more general tool, proved to be the more performant solution in practice due to its highly engineered and optimized solver engine.

5. Code:
Self Explanatory.

Task 2 (MDP Modeling: Strategic Card Play):

Modeling the card game required a direct, high fidelity translation of the game rules into a mathematical MDP. The feasibility of this approach was entirely dependent on the highly efficient planner developed (lp algorithm) in Task 1.

1. **State Space:**
    a. Design: A state was defined as the exact set of cards in the player's hand, including both value and suit (e.g., {3H, 7D}). This is a direct, brute force representation of the game state that preserves all information.
    b. Challenge: This approach results in a state space explosion. The number of possible hands is enormous, growing combinatorially with the number of cards. For a moderate threshold, the number of states can easily reach into the hundreds of thousands or more.
    c. Feasibility: This high fidelity model was only practical because the Modified Policy Iteration solver from Task 1 was fast enough to handle such a large state space within the required time limits. An unoptimized planner would have made this direct modeling approach impossible. Two terminal states, S_STOP and S_BUST, were also added.
2. **Action Space and Transition Function:**
    a. The action space was kept exactly as defined in the problem specification, with 28 actions (0 for *Add*, 1-26 for specific card *Swaps*, 27 for *Stop*).
    b. The set of remaining_deck cards could be determined precisely by taking the set difference between the full 26 card deck and the cards in the current state's hand.
    c. The probability of drawing any specific card was simply $1/|remaining\_deck|$.
    d. For a *Swap* action (e.g., "Swap 5H"), the transition was unambiguous. If the hand contained 5H, the action was valid. If it did not, the action was invalid and resulted in a self loop where it transitioned back to the same state with probability 1.0.
3. **Reward Function and Discount (γ)**
    a. **Rewards:** Rewards are zero for all intermediate transitions. A non zero reward is given only upon entering the S_STOP state. This reward is equal to the final score: the sum of the card values plus the bonus if the special sequence is present.
    b. **Discount Factor (γ):** The discount factor was set to $\gamma = 1.0$. Since the game is episodic and the goal is to maximize the total final score, there is no need to discount future rewards.

4. **Analysis of the Optimal Policy**
    a. The optimal policy is generally risk averse. When the hand sum approaches the threshold (e.g., within 4-5 points), the agent almost always chooses to *Stop*. The potential reward from an *Add* action is outweighed by the high probability of busting and receiving a score of zero.
    b. The bonus value significantly changes the agent's behavior. For a large bonus, the policy will take actions that would otherwise seem suboptimal. For example, it might *Swap* a high value card (e.g., a 10) for a chance to draw a low value card needed to complete the special sequence, even if it temporarily lowers the hand's sum. If the bonus is zero, this behavior is not seen.
    c. The Swap action is used in two primary ways:
        i. To Improve: In the mid game, swapping a low value card for a chance at a higher one is a common, low risk strategy to increase the score.
        ii. To Survive: When the hand sum is high, swapping a high value card is a key defensive move to lower the sum, reduce the risk of busting, and stay in the game longer to pursue other opportunities (like the bonus).

5. **Code:**

```
def generate_states(threshold):
    state_map = {(): 0}  # Maps hand tuple -> state index
    index_map = [()]     # Maps state index -> hand tuple
    queue = deque([()])
    while queue:
        curr_hand = queue.popleft()
        curr_hand_set = set(curr_hand)
        for card in ALL_CARDS:
            if card not in curr_hand_set:
                new_hand_list = sorted(list(curr_hand) + [card])
                new_hand_tuple = tuple(new_hand_list)
                total = sum(c[0] for c in new_hand_tuple)
                if(total<threshold and (new_hand_tuple not in
state_map)):
                    new_index = len(state_map)
                    state_map[new_hand_tuple] = new_index
                    index_map.append(new_hand_tuple)
                    queue.append(new_hand_tuple)
    return state_map, index_map
```

All possible states are made which were possible for a given threshold and each state was mapped to a state number.
Rest of the code is self explanatory.